

UE24CS352A — MACHINE LEARNING MINI-PROJECT

Predicting a Decline in Patient Reported Outcomes for Cancer Patients on Chemotherapy

Problem Statement #8 | Team-15 | SEC K

INTRODUCTION

Team No. 15

Problem Statement #8: Predicting a Decline in Patient Reported Outcomes for Cancer Patients on Chemotherapy

Section: K

Team Member 1: -

Name: Shrujan N

SRN: PES1UG24CS624

Team Member 2: -

Name: Dhanush S Shekhar

SRN: PES1UG24CS662

1. Clinical Context & Objective Overview

- Chemotherapy causes severe physical adverse effects (fatigue, pain, cognitive slowing).
- Clinical blood tests do not measure how patients actually feel day-to-day.
- Patient Reported Outcomes (PROs) measure subjective health using PROMIS Global Physical Health (GPH) scores.
- GPH score population distribution: Mean = 50, Standard Deviation = 10.
- Goal: Predict health decline BEFORE chemotherapy day 1 to enable early supportive interventions.

2. End-to-End Pipeline Architecture

STAGE 1

Dataset Generation

`generate_dataset.py`

Simulates 1,252 patient records with vitals, labs, and targets.

STAGE 2

Time-Decay Featurizer

`featurization.py`

Applies $w_i = \exp(-t/365)$ and builds KNN & Binned matrices.

STAGE 3

Baseline Models

`train_baselines.py`

10-Fold CV on LASSO, Ridge, Elastic Net, and KNN.

STAGE 4

Advanced ML Models

`train_advanced.py`

Trains LightGBM, RF, SVM, and FFNN on 80/20 train/test split.

STAGE 5

Ablation & ROC Plots

`ablation_and_plots.py`

Builds Soft-Voting Ensemble, strips features, and plots ROC curves.

PIPELINE STAGE 1: Synthetic Dataset Generation (generate_dataset.py)

```
#####  
Problem 8 Mini-Project Pipeline  
Problem #8: Predicting a Decline in Patient Reported Outcomes for Cancer Patients on Chemotherapy  
#####  
  
[STEP 1/5] Running Synthetic STARR EHR Data Generator...  
Generating synthetic STARR EHR dataset for 1252 patients...  
✓ Dataset generation complete!
```

Step 1 generates our synthetic STARR cohort of 1,252 cancer patients, creating 6 months of pre-chemotherapy clinical history and target outcome labels while preventing data leakage

- generate_dataset.py uses numpy.random with seed 42 to create 1,252 patient records.
- For each patient, it generates: Baseline PROMIS GPH physical health score (mean = 48.5, std = 9.2).
- 6 months of historical time-series events (5 to 20 visits per patient) containing 7 vitals, 149 lab tests, 291 procedures, 266 diagnoses, 121 medications, and 5 utilization metrics.
- Calculates y_drop (1 if health score drops ≥ 10 points; 0 otherwise) and y_threshold (1 if health score ≤ 40 ; 0 otherwise).
- Saves summary data to data/patients_summary.csv and longitudinal data to data/raw_patients_longitudinal.npy.

PIPELINE STAGE 2: Time-Decay Feature (featurization.py)

```
[STEP 2/5] Running Exponential Time Decay Featurization Engine...  
Executing feature extraction engine with exponential time decay featurization...  
Building KNN Imputed Lab Feature Matrix (933 features)...  
Building Binned Lab Feature Matrix with Missingness (1367 features)...  
✓ Featurization successful!  
KNN Feature Matrix Shape: (1252, 933)  
Binned Feature Matrix Shape: (1252, 1367)
```

Step 2 converts longitudinal medical records into static feature matrices using exponential time-decay weighting $w_i = e^{(-t/365)}$. It builds two competing representations: a 933-feature KNN imputed matrix and a 1,367-feature binned matrix.

- For every patient, featurization.py calculates an exponentially time-decayed average for every lab test and vital sign: $f = \text{sum}(w_i * x_i) / \text{sum}(w_i)$, where $w_i = \exp(-t_i / 365)$, where t_i is the number of days before chemotherapy start date ($t=0$).
- Matrix 1 (KNN Imputed - 933 Features): Uses `KNNImputer(n_neighbors=5)` to fill missing lab values based on Euclidean distance across similar patients.
- Matrix 2 (Binned Labs - 1,367 Features): Splits lab values into 3 tertile bins (T1, T2, T3) plus a 4th explicit binary indicator variable (`is_missing = 1` if missing, `0` if present).
- Saves matrices to `data/X_knn_933.csv`, `data/X_binned_1367.csv`, and `data/y_targets.csv`.
- Time Decay: A lab test done 2 days before chemotherapy (w approx 1.0) is much more important than a test from 5 months ago (w approx 0.61).
- Dual Matrices: Tests whether continuous numerical imputation (KNN) or discrete binning with missingness indicators performs better.

PIPELINE STAGE 3: Baseline Models Evaluation (train_baselines.py)

```
[STEP 3/5] Evaluating Baseline Linear & Distance Models...
Evaluating Baseline Models (LASSO, Ridge, ElasticNet, KNN)...
[KNN Labs (933)] | [10 point drop in GPH] | LASSO      Mean AUROC: 0.9910
[KNN Labs (933)] | [10 point drop in GPH] | Ridge      Mean AUROC: 0.9889
[KNN Labs (933)] | [10 point drop in GPH] | Elastic Net Mean AUROC: 0.9880
[KNN Labs (933)] | [10 point drop in GPH] | KNN        Mean AUROC: 0.9929
[KNN Labs (933)] | [GPH below 40]       | LASSO      Mean AUROC: 0.9860
[KNN Labs (933)] | [GPH below 40]       | Ridge      Mean AUROC: 0.9912
[KNN Labs (933)] | [GPH below 40]       | Elastic Net Mean AUROC: 0.9821
[KNN Labs (933)] | [GPH below 40]       | KNN        Mean AUROC: 0.9943
[Binned Labs (1367)] | [10 point drop in GPH] | LASSO      Mean AUROC: 0.9882
[Binned Labs (1367)] | [10 point drop in GPH] | Ridge      Mean AUROC: 0.9889
[Binned Labs (1367)] | [10 point drop in GPH] | Elastic Net Mean AUROC: 0.9860
[Binned Labs (1367)] | [10 point drop in GPH] | KNN        Mean AUROC: 0.9890
[Binned Labs (1367)] | [GPH below 40]       | LASSO      Mean AUROC: 0.9800
[Binned Labs (1367)] | [GPH below 40]       | Ridge      Mean AUROC: 0.9862
[Binned Labs (1367)] | [GPH below 40]       | Elastic Net Mean AUROC: 0.9827
[Binned Labs (1367)] | [GPH below 40]       | KNN        Mean AUROC: 0.9909

✓ Baseline training completed! Saved outputs/baseline_results.csv
```

PIPELINE STAGE 3: Baseline Models Evaluation (train_baselines.py)

Step 3 runs 10-fold cross-validation on our baseline models—LASSO, Ridge, Elastic Net, and KNN—using standardized scaling to establish linear benchmark AUROC scores.

- Establishes simple linear benchmark scores to determine if complex non-linear models actually improve prediction accuracy.
- Uses 10-fold Stratified Cross-Validation (KFold(n_splits=10)).
- Inside every fold, StandardScaler() scales training data (mean=0, std=1) and fits 4 baseline models:
- 1. LASSO (L1): Regularized logistic regression forcing useless feature weights to zero.
- 2. Ridge (L2): Regularized logistic regression shrinking feature weights smoothly.
- 3. Elastic Net: Combined L1 + L2 regularized logistic regression.
- 4. KNN: Distance-based classifier (K=35).
- Calculates mean AUROC score across all 10 folds and saves to outputs/baseline_results.csv.

PIPELINE STAGE 4: Advanced ML Models Training (train_advanced.py)

[STEP 4/5] Training Advanced Models (LightGBM, SVM, RF, FFNN)...

=== KNN | DROP ===

-> LightGBM
-> Random Forest
-> SVM
-> FFNN

=== KNN | THRESHOLD ===

-> LightGBM
-> Random Forest
-> SVM
-> FFNN

=== BINNED | DROP ===

-> LightGBM
-> Random Forest
-> SVM
-> FFNN

=== BINNED | THRESHOLD ===

-> LightGBM
-> Random Forest
-> SVM
-> FFNN

Advanced-model results saved to outputs/advanced_results.csv

Dataset	Outcome	Model	CV_AUROC_Mean	CV_AUROC_SD	Test_AUROC	Test_Size	Model_File
knn	drop	LightGBM	0.9915	0.0035	0.9914	251	models/knn_drop_lightgbm.joblib
knn	drop	Random Forest	0.9940	0.0025	0.9923	251	models/knn_drop_random_forest.joblib
knn	drop	SVM	0.9909	0.0031	0.9901	251	models/knn_drop_svm.joblib
knn	drop	FFNN	0.9899	0.0049	0.9786	251	models/knn_drop_ffnn.joblib
knn	threshold	LightGBM	0.9895	0.0027	0.9902	251	models/knn_threshold_lightgbm.joblib
knn	threshold	Random Forest	0.9931	0.0024	0.9950	251	models/knn_threshold_random_forest.joblib
knn	threshold	SVM	0.9901	0.0037	0.9936	251	models/knn_threshold_svm.joblib
knn	threshold	FFNN	0.9855	0.0053	0.9847	251	models/knn_threshold_ffnn.joblib
bin	drop	LightGBM	0.9834	0.0060	0.9651	251	models/bin_drop_lightgbm.joblib
bin	drop	Random Forest	0.9930	0.0022	0.9850	251	models/bin_drop_random_forest.joblib
bin	drop	SVM	0.9903	0.0045	0.9870	251	models/bin_drop_svm.joblib
bin	drop	FFNN	0.9859	0.0078	0.9882	251	models/bin_drop_ffnn.joblib
bin	threshold	LightGBM	0.9758	0.0058	0.9811	251	models/bin_threshold_lightgbm.joblib

PIPELINE STAGE 4: Advanced ML Models Training (train_advanced.py)

Step 4 trains our advanced models—LightGBM, Random Forest, SVM, and Neural Networks—evaluating performance on a held-out test split and saving binary model weights into models/

- Non-linear models capture multi-variable clinical interactions (e.g., high blood pressure combined with low kidney function) that linear models miss.
- Splits data into an 80% training set and a 20% held-out testing set (train_test_split).
- Fits 4 advanced non-linear algorithms:
 1. LightGBM: Gradient boosted decision trees built sequentially to minimize log-loss.
 2. Random Forest: 400 independent decision trees voting together.
 3. SVM: Support Vector Classifier with RBF kernel maximizing hyper-plane decision margins.
 4. FFNN: Multi-Layer Perceptron Neural Network (128 -> 32 nodes) with Adam optimizer and early stopping.
- Evaluates on held-out test split and serializes trained model objects into models/*.joblib.
- Saves output summary to outputs/advanced_results.csv.

PIPELINE STAGE 5: Ablation & ROC Generation (ablation_and_plots.py)

[STEP 5/5] Generating ROC Curves & Ablation Results...

Selected ensemble: ['Ridge', 'Random Forest', 'SVM']

Ablation results:

Feature_Set	Removed_Group	N_Features	Test_AUROC	Selected_Models	AUROC_Change_vs_Full
Full model	None	933	0.9913	Ridge, Random Forest, SVM	0.0000
Without Demographics	Demographics	925	0.9914	Ridge, Random Forest, SVM	0.0001
Without Utilization + baseline	Utilization + baseline	927	0.9913	Ridge, Random Forest, SVM	0.0000
Without Vitals	Vitals	926	0.9912	Ridge, Random Forest, SVM	-0.0001
Without Procedures	Procedures	642	0.9902	Ridge, Random Forest, SVM	-0.0011
Without Diagnoses	Diagnoses	667	0.9937	Ridge, Random Forest, SVM	0.0024
Without Medications	Medications	812	0.9920	Ridge, Random Forest, SVM	0.0007
Without Labs	Labs	784	0.9454	Ridge, Random Forest, SVM	-0.0459

Saved:

outputs/ablation_results.csv
 outputs/ablation_model_selection.csv
 outputs/roc_curves.png

✓ Pipeline execution finished in 167.31 seconds

PIPELINE STAGE 5: Ablation & ROC Generation (ablation_and_plots.py)

Step 5 builds our Soft-Voting Ensemble model and performs feature ablation by stripping away feature categories one by one. It proves laboratory histories carry primary predictive signal and renders our ROC curves into outputs/roc_curves.png

- Ablation: Proves WHICH feature categories actually drive clinical predictions. Removing Labs caused the biggest drop (AUROC -> 0.9454), proving laboratory trends carry critical diagnostic weight.
- 1. Selects top constituent models (Logistic + Tree + SVM) to build a Soft-Voting Ensemble.
- 2. Evaluates full ensemble on test set.
- 3. Removes one feature group at a time (Demographics, Vitals, Labs, etc.), retrains the ensemble, and measures the change in AUROC (delta AUROC).
- 4. Renders ROC curves using matplotlib and saves to outputs/roc_curves.png.